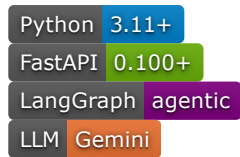


ABOIA — Autonomous Business Operations Intelligence Agent



Enterprise Safety & Evaluation Report

For a deeper technical and operational analysis of ABOIA — including governance architecture, anomaly detection methodology, SLA lifecycle design, escalation policies, and enterprise scalability considerations — refer to the full [Technical Report](#).

Overview

ABOIA is a governed agentic framework that transforms e-commerce KPI anomalies into prioritized, auditable, and operationally safe actions.

It combines:

- Statistical anomaly detection
- LLM-based root cause reasoning (Gemini)
- Deterministic validation guardrails
- Risk-driven prioritization and SLA enforcement
- Human-in-the-loop approval gates
- Structured lifecycle execution with full observability

Rather than allowing the LLM to directly control execution, ABOIA separates probabilistic reasoning from deterministic governance. This enables the system to validate, prioritize, and safely route operational actions while preventing unsafe automation caused by unreliable or low-confidence AI reasoning.

The system is demonstrated using the Olist Brazilian E-Commerce Dataset, where raw transactional records are aggregated into daily business KPIs for anomaly detection, reasoning, and operational action simulation.

Simulation Control

API Key

Start Date

2018/08/26

End Date

2018/09/01

Initialize Simulation

Run Next Day

Backend Connected

Navigation

Timeline

Episode Deep Dive

Pending Approvals

SLA Monitor

System Metrics

Action Execution

Trigger Execution Loop



ABOIA – Governed AI Operations Console



Episode Timeline

	Date	Risk	Priority	Governance Score	AI Validation	Actions
0	2018-08-29	● HIGH	P0	● 95	✔ PASSED	3
1	2018-08-28	● LOW	P1	● 71	✗ FAILED	1
2	2018-08-27	● LOW	P2	● 78	⚠ WARNING	1



Why ABOIA Is Different

- Separates probabilistic LLM reasoning from deterministic governance and policy enforcement
- Enforces structured lifecycle transitions (approval → execution → SLA resolution)
- Prevents unsafe automation through governance-based escalation overrides
- Ensures idempotent and replay-safe action execution
- Maintains a complete audit trail across the operational lifecycle



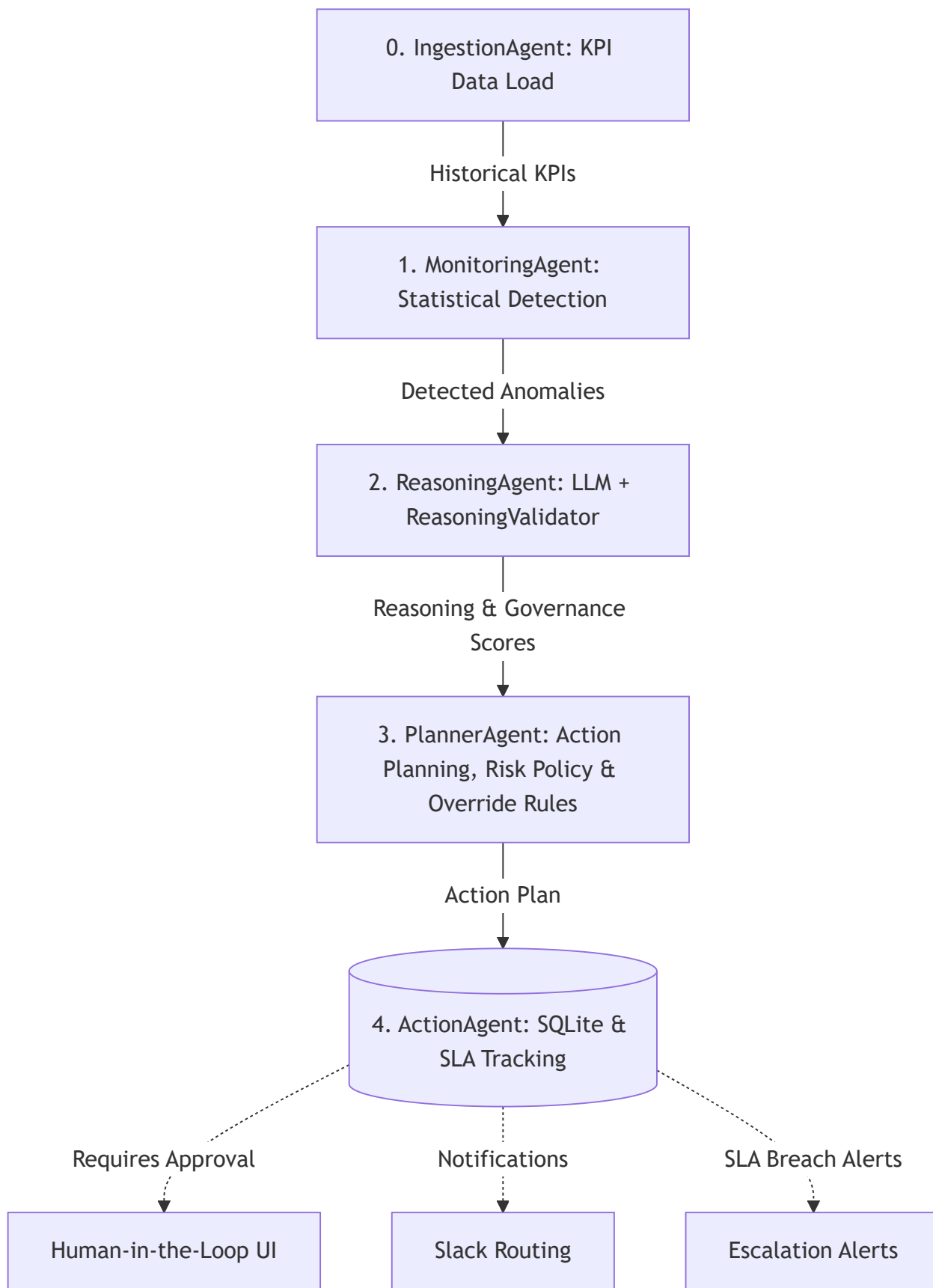
Table of Contents

- [Architecture](#)
- [Agent Pipeline](#)
- [Key Features](#)
- [Action Lifecycle State Machine](#)
- [Approval & Risk Policy](#)
- [Config-Driven Policy](#)
- [LLM Reliability & Safety Design](#)
- [Agent Data Contracts](#)
- [Project Structure](#)
- [Setup & Installation](#)
- [Running the System](#)
- [API Reference](#)
- [Notification System](#)

- [Debugging & Observability](#)
- [Tech Stack](#)
- [Summary](#)

Architecture

ABOIA is built as three loosely coupled components: a FastAPI backend, a Streamlit dashboard, and a SQLite persistence layer for state tracking and lifecycle management. The core workflow is orchestrated using LangGraph:



The system operates as a daily batch simulation. Each execution processes KPIs for a single simulated date, persists the resulting state, and then advances to the next day. This ensures chronological isolation, preventing agents from accessing future telemetry data during anomaly detection and reasoning.

Agent Pipeline

Step / Node	Agent / Component	Inputs	Primary Outputs / Artifacts	Key Responsibility
0	DataIngestionAgent	Raw CSV transaction logs in data/	daily_kpis (Pandas DataFrame)	Loads and aggregates KPI data while enforcing timeline isolation and warmup rules.
1	MonitoringAgent	daily_kpis + historical KPI data	metrics_df + anomaly episodes	Runs statistical anomaly detection, seasonal analysis, and cross-metric correlation checks.
2	ReasoningAgent	anomalies + metrics_df	Structured LLM reasoning output + governance scorecard	Generates structured root-cause reasoning and validates responses using deterministic governance checks.
3	PlannerAgent	Reasoning output + Validation results	Persisted action_plan records	Maps diagnoses to predefined operational actions and applies governance override rules.
4	ActionAgent	action_plan	Ticket state updates + Slack notifications	Handles approvals, execution lifecycle management, and SLA tracking and escalation workflows.

Key Features

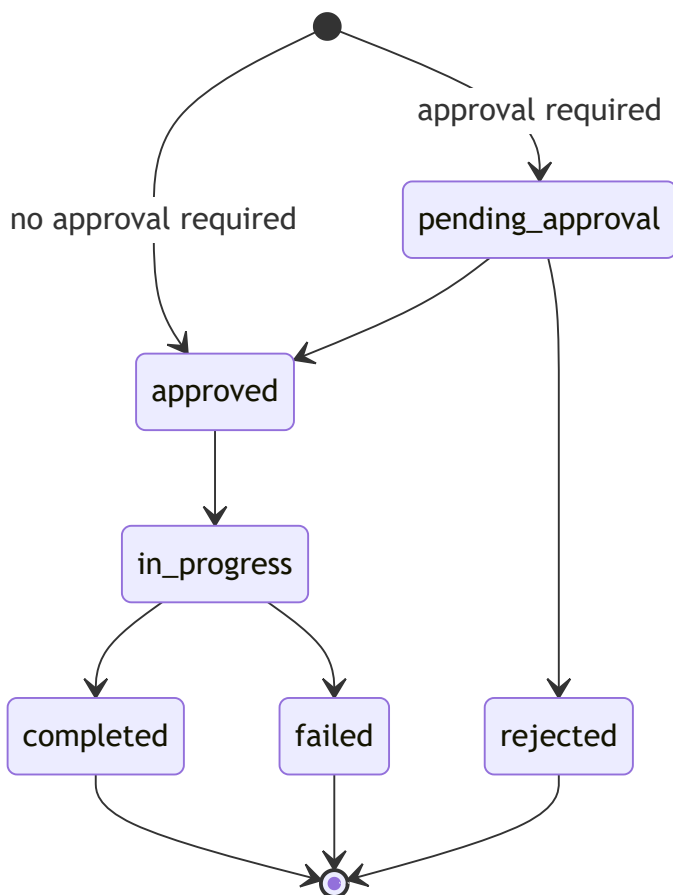
- Multi-method anomaly detection:** Rolling z-score, percentage change, IQR (global), EWMA, seasonal, and cross-metric correlation checks. Z-score and percentage-change thresholds are **auto-**

tuned per metric using the 95th percentile of historical values.

- **LLM reasoning with deterministic guard-rails:** Gemini generates root cause and risk level assessments, while the `ReasoningValidator` evaluates outputs across four governance dimensions (**Metric Grounding**, **Analytical Depth**, **Risk Policy Alignment**, and **Confidence Parity**).
- **Three-tier priority system (P0/P1/P2)** driven by risk level, governance validation and escalation rules.
- **Human-in-the-loop approval gate:** High-risk or high-uncertainty actions require approval from a Business Head before execution.
- **Slack notification routing:** based on owner team (Marketing, Product, Engineering, Operations) and operational event type.
- **Full audit trail:** through structured NDJSON debug logs for every pipeline stage (`debug_output/`).
- **Idempotent lifecycle execution:** Re-running the lifecycle worker never re-executes completed actions or mutates terminal execution states.

Action Lifecycle State Machine

Execution Lifecycle:



Simulation Control

API Key

.....

👁

Start Date

2018/08/26

End Date

2018/09/01

🚀 Initialize Simulation

▶ Run Next Day

Backend Connected

Navigation

Timeline

🔍 Episode Deep Dive

📋 Pending Approvals

🕒 SLA Monitor

📊 System Metrics

Action Execution

⚙ Trigger Execution Loop

SIM-2018-08-27

▼

🚦 Episode Lifecycle

🚨 Detected

🧠 Analyzed

📅 Planned

✅ Approvals Done

✅ Execution Resolved

> 🧠 AI Reasoning

> ❤ Governance Evaluation

> 📉 Episode Metric Plots

▼ 📅 Action Plan

monitoring | 🚨 P2

Description: Monitor metrics for next 48 hours

Owner: Operations Team

Status: Completed

👤 Approval Required: None (NOT_REQUIRED)

Simulation Control

API Key

.....

👁

Start Date

2018/07/17

End Date

2018/07/19

🚀 Initialize Simulation

▶ Run Next Day

Backend Connected

Navigation

Timeline

🔍 Episode Deep Dive

📋 Pending Approvals

🕒 SLA Monitor

📊 System Metrics

Action Execution

⚙ Trigger Execution Loop

🚀 ABOIA – Governed AI Operations Console

📄 Pending Approvals Inbox

🚨 P0 | Investigate traffic sources and ad campaigns

Action ID: [SIM-2018-07-18-traffic](#)

Owner: Marketing Team

SLA Deadline: 2018-07-18T04:00:00

✅ Approve

❌ Reject

🚨 P0 | Review checkout funnel and recent UI changes

Action ID: [SIM-2018-07-18-conversion](#)

Owner: Product Team

SLA Deadline: 2018-07-18T04:00:00

✅ Approve

❌ Reject

🚨 P0 | Investigate platform stability and infrastructure health

Action ID: [SIM-2018-07-18-platform](#)

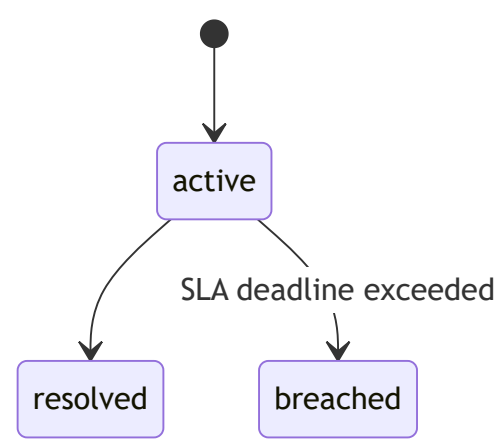
Owner: SRE Team

SLA Deadline: 2018-07-18T04:00:00

✅ Approve

❌ Reject

SLA tracking:



- SLA tracking is maintained independently from execution state transitions.
- `rejected` , `failed` , and `completed` are terminal execution states.
- `Completed`, `rejected`, and `failed` actions all resolve SLA tracking.
- The lifecycle worker is idempotent and replay-safe.

Simulation Control

API Key

.....

Start Date

2018/07/17

End Date

2018/07/19

Initialize Simulation

Run Next Day

Backend Connected

Navigation

- Timeline
- Episode Deep Dive
- Pending Approvals
- SLA Monitor**
- System Metrics

Action Execution

Trigger Execution Loop

ABOIA – Governed AI Operations Console

SLA Monitoring

	Episode	Action	Owner	Priority	SLA Status	Status
0	SIM-2018-07-18	SIM-2018-07-18-traffic	Marketing Team	P0	● ACTIVE	Pending Approval
1	SIM-2018-07-18	SIM-2018-07-18-conversion	Product Team	P0	● ACTIVE	Pending Approval
2	SIM-2018-07-18	SIM-2018-07-18-platform	SRE Team	P0	● ACTIVE	Pending Approval

Deploy

Base Policy (Risk → Priority)

Risk Level	Priority	SLA	Requires Approval
high	P0	4 hours	 Always (Business Head)
medium	P1	8 hours	 Default No
low	P2	24 hours	 Default No

Simulation Control

API Key

👁

Start Date

2018/08/26

End Date

2018/09/01

🔥 Initialize Simulation

▶ Run Next Day

Backend Connected

Navigation

Timeline

🔍

Episode Deep Dive

⏸

Pending Approvals

📊

SLA Monitor

📈

System Metrics

Action Execution

🔧

Trigger Execution Loop

Deploy⋮

> AI Reasoning

▼ Governance Evaluation

Overall Governance Score:

●

 95

Evaluation Reasoning

▼ 🔍 Grounding Score: 100

Excellent metric grounding. The reasoning explicitly references the affected database metrics: ['orders', 'unique_users', 'gmv', 'items_sold', 'aov', 'orders_per_user', 'conversion_rate'].

▼ 🎯 Risk Alignment Score: 100

Perfect alignment. The generated risk level 'high' appropriately matches the structural episode severity of 'high'.

▼ 📊 Confidence Gap Score: 95

Acceptable confidence variance. AI certainty (95%) is reasonably close to the statistical anomaly strength (90%).

▼ 🧠 Analytical Depth Score: 85

High analytical depth achieved. Detected advanced reasoning signals: percentage_change, baseline_reference, multi_metric_context, narrative_depth.

> 📉 Episode Metric Plots

> 📋 Action Plan

Confidence Signals

ABOIA maintains two independent confidence signals used during governance evaluation and escalation decisions:

Signal	Source	Range	Description
anomaly_confidence	Deterministic (system)	0–100	Statistical signal strength derived from anomaly severity, metric overlap, and affected KPIs
reasoning_confidence	LLM (Gemini)	0–100	LLM's self-reported confidence in the generated root-cause analysis

A large divergence between these signals (for example, strong statistical anomalies combined with low LLM confidence) triggers escalation to manual approval.

Governance Escalation & Priority Overrides

When governance checks fail or confidence signals become unreliable, the `PlannerAgent` applies escalation rules that increase ticket priority and require human approval before execution:

Escalation Override Rule	Trigger Condition	Governance Action
Critical Validation Override	<code>validation_severity == "critical"</code>	Priority escalated by one level + <code>requires_approval = True</code>
Confidence Gap Override	<code>abs(anomaly_confidence - reasoning_confidence) > 50</code>	<code>requires_approval = True</code>
Unrecognized Playbook Override	No matching action mapping found	Priority escalated by one level + <code>requires_approval = True</code>
Low Overall Reasoning Override	<code>overall_reasoning_score < 50</code>	Priority escalated by one level + <code>requires_approval = True</code>
Weak Metric Grounding Override	<code>grounding_score < 40</code>	<code>requires_approval = True</code>
Risk Misalignment Override	<code>risk_alignment_score < 60</code>	Priority escalated by one level + <code>requires_approval = True</code>

Configuration-Driven Orchestration

To keep operational logic separate from application code, KPI definitions, monitoring rules, governance policies, and action mappings are externalized through YAML configuration files under `config/` :

- `kpis.yaml` : Defines KPI aggregation rules, source columns, and derived metric formulas (e.g., `conversion_rate = orders / visits`).

- `monitoring.yaml` : Specifies the KPI metrics monitored by the anomaly detection pipeline.
- `risk_policy.yaml` : Stores risk-to-priority mappings, SLA targets, approval requirements, governance score weights, and escalation thresholds.
- `root_cause_action_map.yaml` : Maps LLM-generated root-cause keywords to predefined operational actions, ticket types, and owner teams.

LLM Reliability & Safety Design

The LLM integration layer is designed with reliability and safety safeguards to prevent workflow instability:

- Configurable timeout enforcement
- Automatic retries for transient failures (up to 3 total attempts)
- Deterministic JSON extraction from model output
- Structured fallback responses on repeated failure
- NDJSON debug logging for prompts and model outputs

If the LLM fails, the system safely degrades without interrupting the deterministic monitoring and governance pipeline.

This ensures the AI reasoning layer enhances — but never destabilizes — operational automation.

Agent Data Contracts

Key input/output structures flowing through the pipeline:

Monitoring Output — Anomaly Record

```
{  
  "date": "2024-01-15",  
  "metric": "visits",  
  "value": 1823.0,  
  "type": "zscore",  
  "score": 3.4  
}
```

One record is generated per detected anomaly. Multiple monitoring methods may flag the same (date, metric) pair — these are deduplicated before episode construction.

Reasoning Output — LLM Result

```
{  
  "root_cause": "Traffic drop from paid campaigns",  
  "business_impact": "Revenue at risk due to reduced funnel entry",  
  "risk_level": "high",  
  "anomaly_confidence": 90,  
  "reasoning_confidence": 85  
}
```

Generated by Gemini and validated by ReasoningValidator before influencing downstream planning decisions.

Action Plan — Planner Output

```
{
  "generated_at": "2024-01-15T10:00:00",
  "risk_level": "high",
  "priority": "P0",
  "actions": [
    {
      "action_id": "uuid",
      "type": "investigation",
      "description": "Investigate traffic sources and ad campaigns",
      "owner": "Marketing Team",
      "priority": "P0",
      "sla_hours": 4,
      "requires_approval": true,
      "approval_role": "Business Head",
      "status": "pending"
    }
  ],
  "decision_trace": {
    "risk_level": "high",
    "anomaly_confidence": 90,
    "reasoning_confidence": 85,
    "validation_severity": "none",
    "confidence_gap": 5,
    "escalation_flags": [],
    "matched_keywords": ["traffic"]
  }
}
```

Produced deterministically from `risk_level` and governance escalation rules. No LLM involvement occurs at this stage.

Project Structure

```
ABOIA/
├─ app/
│  ├─ agents/ # Core agent pipeline (ingestion, monitoring, reasoning, planning, execution)
│  ├─ graph/ # LangGraph workflow orchestration
│  ├─ api/ # FastAPI route handlers
│  ├─ services/ # LLM integration, validation, notifications, logging, auth
│  ├─ db/ # SQLite models and persistence layer
│  ├─ utils/ # KPI aggregation and validation utilities
│  ├─ main.py # FastAPI entrypoint
│  └─ state.py # Shared LangGraph AgentState
├─
├─ config/
│  ├─ kpis.yaml
│  ├─ monitoring.yaml
│  ├─ risk_policy.yaml
│  └─ root_cause_action_map.yaml
├─
├─ data/ # Raw dataset and generated KPI windows
├─ debug_output/ # NDJSON pipeline traces and validation logs
├─ logs/ # Application logs
├─ .env.example # Environment variable template
└─ requirements.txt
```

Setup & Installation

Prerequisites

- Python 3.11+
- A Gemini API key (from [Google AI Studio](#))
- (Optional) Slack Bot Token for notification routing

1. Clone the Repository & Create a Virtual Environment

```
git clone <your-repo-url>
cd ABOIA
python -m venv venv
# Windows
venv\Scripts\activate
# Linux/macOS
source venv/bin/activate
```

2. Install Dependencies

```
pip install -r requirements.txt
```

3. Configure Environment Variables

```
cp .env.example .env
```

Edit .env :

```
# Required
API_KEY=your_api_key_here

GEMINI_API_KEY=your_gemini_api_key_here
GEMINI_MODEL=model_name_here

# Slack notifications
SLACK_BOT_TOKEN=xoxb-your-bot-token
SLACK_MARKETING_CHANNEL=#marketing-alerts
SLACK_PRODUCT_CHANNEL=#product-alerts
SLACK_ENGINEERING_CHANNEL=#engineering-alerts
SLACK_OPERATIONS_CHANNEL=#operations-alerts
SLACK_APPROVAL_CHANNEL=#approvals
SLACK_ALERT_CHANNEL=#sla-alerts
```

4. Add Dataset Files

The project was developed and evaluated using the **Olist Brazilian E-Commerce Dataset**. Raw transactional CSV files should be placed inside `data/raw/`. The ingestion pipeline aggregates these records into daily KPI windows for downstream monitoring and reasoning.

Running the System

ABOIA can be executed either through the interactive Streamlit dashboard (recommended for visualization and approvals) or directly through REST API endpoints.

Option A: Streamlit Dashboard

1. Start the FastAPI Backend:

```
uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload
```

2. Start the Streamlit Frontend (in a separate terminal):

```
streamlit run streamlit_app/dashboard.py
```

3. Open the local dashboard URL displayed in the terminal (typically `http://localhost:8501`).

Option B: REST API Execution

1. Start the FastAPI Backend

```
uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload
```

2. Run the Agent Pipeline for a Simulated Day

```
curl -X POST http://127.0.0.1:8000/v1/run_day \
  -H "x-api-key: your_api_key_here" \
  -H "Content-Type: application/json" \
  -d '{"start_date": "2018-08-27", "end_date": "2018-08-27"}'
```

This executes the full workflow for a single simulated day:

Ingestion → Monitoring → Reasoning → Planning → Action

3. Run the Lifecycle Worker

```
curl -X POST http://127.0.0.1:8000/v1/system/run_lifecycle \
  -H "x-api-key: your_api_key_here"
```

Use this endpoint after approving actions to process execution workflows, lifecycle transitions, and SLA auditing.

Execution Model

The pipeline currently executes synchronously inside the HTTP request lifecycle. This design preserves deterministic execution semantics and simplifies debugging for development and demonstration environments.

In production deployments, the workflow can be offloaded to background workers (e.g., task queues or asynchronous executors) to prevent long-running LLM calls from blocking request threads.

The architecture cleanly separates orchestration, state transitions, and lifecycle execution, making future migration to asynchronous execution straightforward without requiring changes to core business logic.

API Reference

The FastAPI backend exposes REST endpoints for pipeline execution, approvals, lifecycle orchestration, SLA monitoring, and observability.

Core Endpoint Groups

Category	Example Endpoints
Pipeline Execution	POST /v1/run_day , POST /v1/run_simulation
Action Management	GET /v1/actions/ , GET /v1/actions/{action_id}
Approvals	GET /v1/approvals/pending , POST /approve , POST /reject
Episodes & Reasoning	GET /v1/episodes/
SLA Monitoring	GET /v1/sla/breached , GET /v1/sla/active
System Operations	GET /v1/system/health , POST /v1/system/run_lifecycle

Example Queries

GET /v1/actions/?status=pending_approval
GET /v1/actions/?priority=P0&owner=Marketing Team
GET /v1/actions/?sla_status=breached

API Design Notes

- All public endpoints are versioned under `/v1`
- List endpoints support `limit` and `offset` pagination
- State-changing routes require `x-api-key` authentication
- Read-only observability endpoints remain public for monitoring use cases

Interactive API docs

`http://127.0.0.1:8000/docs`

Notification System

Slack notifications are dispatched for the following operational events:

Event	Notification Routing
created	Owner team channel
approval_required	Owner team channel + centralized approval channel
approved	Owner team channel + centralized approval channel
rejected	Owner team channel + centralized approval channel
completed	Owner team channel
failed	Owner team channel
sla_breached	centralized alert channel

If `SLACK_BOT_TOKEN` is not configured, notification delivery is safely skipped without interrupting workflow execution.

Debugging & Observability

Structured Logging

All agent activity is logged to both `logs/system.log` and the application console:

Set LOG_LEVEL=DEBUG in .env to enable verbose execution traces.

NDJSON Debug Traces

Each pipeline stage emits structured debug artifacts to `debug_output/` for observability and replay analysis:

File	Description
daily_kpis.csv	Aggregated KPI windows
anomalies.json	Raw anomaly signals
deduplicated_anomalies.json	Consolidated anomaly episodes
episode_reasoning.ndjson	Per-episode reasoning summaries and outputs
planner_agent.ndjson	Final action plans and escalation traces
llm_calls.ndjson	LLM prompts, raw outputs, and parsed responses
reasoning_validation.ndjson	Governance validation results
node_state.ndjson	LangGraph node input/output state snapshots

Tech Stack

- Python 3.11+
- FastAPI
- LangGraph
- SQLAlchemy + SQLite (development environment)
- Gemini (Google AI Studio)
- Slack Bot API
- YAML-driven configuration for KPIs, monitoring rules, governance policies, and action mappings

Summary

ABOIA is a governed agentic operations framework that combines statistical anomaly detection, LLM-backed root-cause reasoning, and deterministic governance to transform business KPI anomalies into structured operational workflows.

The system emphasizes safe automation through human-in-the-loop approvals, governance-based escalation policies, lifecycle-aware execution, and end-to-end observability across every decision stage.